



USDT Native Mesh

Security Assessment

June 28th, 2025 — Prepared by OtterSec

Robert Chen

r@osec.io

Samuel Bétrisey

sam@osec.io

Nicholas R. Putra

nicholas@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	3
Findings	4
Vulnerabilities	5
OS-UNM-ADV-00 Type Confusion in Credits	6
OS-UNM-ADV-01 Flaw in Message Type Handling	7
OS-UNM-ADV-02 Incorrect Compose Message Serialization	8
General Findings	9
OS-UNM-SUG-00 Potential DOS via Front-Running	10
OS-UNM-SUG-01 Code Maturity	11
Appendices	
Vulnerability Rating Scale	12
Procedure	13

01 — Executive Summary

Overview

LayerZero Labs engaged OtterSec to assess the updated `usdt-native-mesh` program. This assessment was conducted between June 11th and June 19th, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning the send credits function, which utilizes inconsistent parsing and encoding logic for credits, allowing malicious input to send credits without reducing the local balance. ([OS-UNM-ADV-00](#)). Additionally, sending OFT only utilizes `SEND_OFT` as the message type, resulting in compose payloads to be ignored on the destination chain, where only `SEND_OFT_AND_CALL` triggers compose execution ([OS-UNM-ADV-01](#)). Furthermore, when encoding OFTSend as a compose message, the function incorrectly splits `lzReceiveGasBottom` across two cells, which may create serialization failures if the original gas value exceeds 2^{24} ([OS-UNM-ADV-02](#)).

We also highlighted a potential denial-of-service scenario via front-running ([OS-UNM-SUG-00](#)), and advised to handle the case where there is a failure in the hop to prevent users from paying twice for a single operation ([OS-UNM-SUG-01](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/LayerZero-Labs/x-usdtoft>. This audit was performed against [ad75806](#) on the EVM code, and against [ecc5fc8](#) on the TON code.

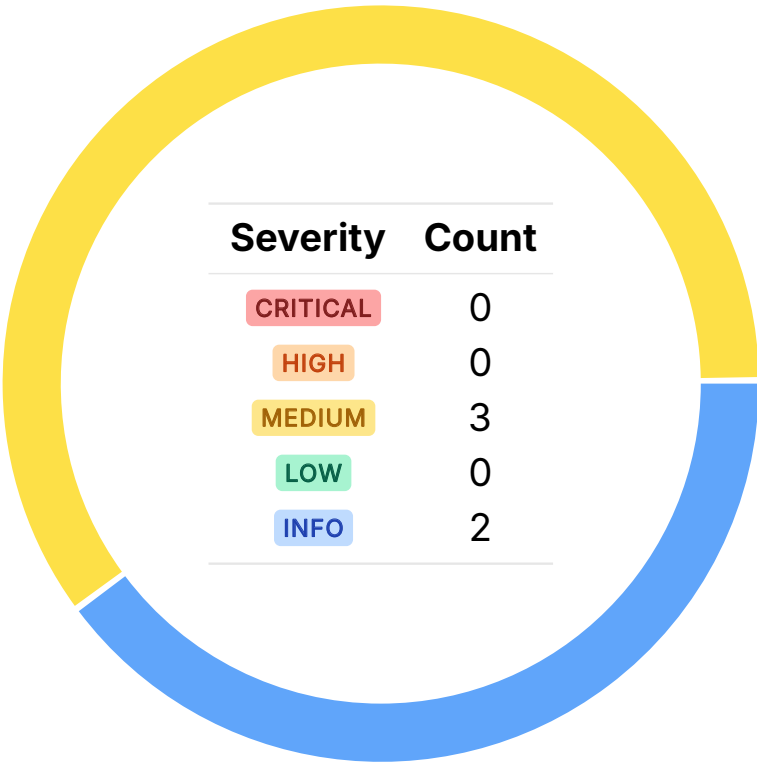
A brief description of the program is as follows:

Name	Description
usdtoft-evm	The module was updated to support LZCompose actions, enabling bi-directional transfers between the AdapterMesh (NativeMesh) and OFTMesh (USDT0 Mesh).
composer	It facilitates cross-mesh token transfers between ADAPTER_MESH and OFT_MESH in the USDTOFT system.
usdtoft-ton	The contracts were updated with the implementation of compose payload and quoting capabilities, and improved fee handling for better automation.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-UNM-ADV-00	MEDIUM	RESOLVED ✓	<code>sendCredits</code> utilizes inconsistent parsing and encoding logic for credits, allowing malicious input to send credits without reducing the local balance.
OS-UNM-ADV-01	MEDIUM	RESOLVED ✓	<code>sendOFT</code> always utilizes <code>SEND_OFT</code> as the message type, resulting in compose payloads to be ignored on the destination chain, where only <code>SEND_OFT_AND_CALL</code> triggers compose execution.
OS-UNM-ADV-02	MEDIUM	RESOLVED ✓	<code>_encodeOftSendAsComposeMessage</code> incorrectly splits <code>lzReceiveGasBottom</code> across two cells, which may create serialization failures if the original gas value exceeds 2^{24} .

Type Confusion in Credits MEDIUM

OS-UNM-ADV-00

Description

In `sendCredits`, the `$outgoingCredits` object is handled inconsistently; it is parsed using `Credits::subtract` (which relies on type metadata), but then encoded utilizing `Credits::getAllCredits` (which assumes fixed field offsets). This mismatch allows an attacker to craft a `$outgoingCredits` cell that appears valid to `subtract` (and results in no actual deduction), while supplying forged values to `getAllCredits` for encoding. As a result, the contract may send credits to a remote chain without decreasing its local credit balance.

```
>_ x-usdtoft/packages/usdtoft-ton/src/usdtOFT/handlerUsdt.fc
```

func

```
tuple sendCredits(cell $OFTCredits) impure inline {
  (cell $storage, tuple actions) = preamble();
  [...]
  ;; Contains a terminal action, thus must be at the end of the actions list
  actions = _lzSend(
    actions,
    dstEid,
    SEND_CREDITS,
    nativeFee,
    zroFee,
    $ExtraOptions,
    _encodeSendCredits(
      $outgoingCredits
    ),
    _encodeSendCreditsCallback(
      dstEid,
      $outgoingCredits
    )
  );
  [...]
  return actions;
}
```

Remediation

Ensure that both `subtract` and `getAllCredits` utilize the same parsing logic, or sanitize the object before usage.

Patch

Resolved in [9f3421e](#).

Flaw in Message Type Handling MEDIUM

OS-UNM-ADV-01

Description

In `handlerUsdt::sendOFT`, the message type (`msgType`) is hardcoded to `SEND_OFT` even when a `composeMessage` is provided. On the EVM destination chain, `UsdtOFT::_lzReceive` only processes composed payloads if the message type is `SEND_OFT_AND_CALL`. As a result, the compose payload is silently ignored, and the expected post-transfer action never executes, despite a successful token transfer.

```
>_ x-usdtoft/packages/usdtoft-ton/src/usdtOFT/handlerUsdt.fc
```

func

```
tuple sendOFT(cell $OFTSend) impure inline {
  [...]
  try {
    (int amountSent, int fee) = _getAmountAndFee($storage, totalAmount);
    (
      int dstEid,
      int nativeFee,
      cell $ExtraOptions
    ) = $OFTSend.OFTSend::getDstEidAndNativeFeeAndExtraOptions();
    [...]
    ;; Contains a terminal action thus must be last in the actions list before returning
    ;; Send the tokens to the remote chain
    actions = _sendOFT(
      actions,
      $OFTSend,
      SEND_OFT,
      amountSent,
      fee,
      composeMessage
    );
    [...]
  } catch(x, n) {
    errorCode = n;
  }
  [...]
}
```

Remediation

Dynamically set the message type based on whether a compose payload is present.

Patch

Resolved in [9f3421e](#).

Incorrect Compose Message Serialization

MEDIUM

OS-UNM-ADV-02

Description

`_encodeOftSendAsComposeMessage` in `usdtOFT/coders` splits the `lzReceiveGas` value into two parts, 104-bit top value and 152-bit bottom value, because the `l` parameter of `splitBits` is set to 256. It then tries to serialize this value with `.store_uint(lzReceiveGasBottom, 24)`. However, if `lzReceiveGas` exceeds 2^{24} , the lower portion may not fit within 24 bits, resulting in a runtime failure during serialization.

Additionally, `optionsLength` is set to `0x50`, while the EVM implementation produces a payload with this field set to `0x5a`.

```
>_ x-usdtoft/packages/usdtoft-ton/src/usdtOFT/coders.fc
```

func

```
cell _encodeOftSendAsComposeMessage(  
  int sender,  
  int dstEid,  
  int to,  
  cell $extraOptions  
) impure inline method_id {  
  [...]   
  ;; we need to split lzReceiveGas because the integer won't fit in the cell it's stored in  
  int cBits = MAX_CELL_WHOLE_BYTE_BITS - (yBits + 256 + 256 + 256 + 16 + 32); ;; 104  
  int dBits = 128 - cBits; ;; 24  
  (int lzReceiveGasTop, int lzReceiveGasBottom) = splitBits(lzReceiveGas, 256, cBits);  
  [...]   
}
```

Remediation

Validate that `lzReceiveGas` fits in the 128-bit field and call `splitBits` with `128` instead of `256`.

Fix the value of `optionsLength`.

Patch

Resolved in [97d32b4](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-UNM-SUG-00	A griefier may front-run <code>lzCompose</code> with zero <code>msg.value</code> , forcing user messages to fail and be retried at the user’s expense, undermining UX and introducing a denial-of-service risk.
OS-UNM-SUG-01	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

Potential DOS via Front-Running

OS-UNM-SUG-00

Description

`lzCompose` in `MultiHopComposer` utilizes a try-catch pattern without validating `msg.value`, enabling a greifer to front-run by calling `endpoint.lzCompose` with zero `msg.value`, resulting in the compose message to become a `failedMessage`, while still marking `lzCompose` as successful. As a result, users must manually retry or request refunds, incurring additional fees. This creates a denial-of-service vector and undermines the improved UX intended for seamless cross-mesh fund transfers.

Remediation

Re-evaluate the updated logic to determine whether the introduced risk is acceptable.

Patch

The issue was acknowledged.

Code Maturity

OS-UNM-SUG-01

Description

1. Currently, due to a failure case in the hop, users will lose their `msg.value` sent to the `MultiHopComposer` since it is not refunded. On calling `retry` or `refund`, users must send additional `msg.value`, effectively paying twice for a single operation. This behavior should be explicitly addressed either via protocol documentation or a design change to improve refundability.
2. `MultiHopComposer` sets `minAmountLD = 0` during hops, disabling slippage protection. This assumes no further deductions will occur after the first leg, but if the second-hop OFT applies additional fees, users may receive less than expected.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in [1d00356](#).
2. Issue #2 was acknowledged.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.